

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ  
ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение  
высшего образования «Санкт-Петербургский политехнический университет  
Петра Великого»

Институт компьютерных наук и кибербезопасности  
Высшая школа технологий искусственного интеллекта  
Направление: 02.03.01 Математика и компьютерные науки

ОТЧЕТ ПО КУРСОВОЙ РАБОТЕ  
по дисциплине Дискретная математика

**Калькулятор «большой» конечной арифметики**  
Вариант 1

Студент,  
группы 5130201/40003

\_\_\_\_\_ Адиатуллин Т.Р

Доцент

\_\_\_\_\_ Востров А.В

«\_\_\_\_\_» \_\_\_\_\_ 2026 г.

Санкт-Петербург, 2026

# Содержание

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| <b>Введение</b>                                                     | <b>4</b>  |
| <b>1 Математическое описание</b>                                    | <b>5</b>  |
| 1.1 Алгебраические структуры                                        | 5         |
| 1.2 Таблицы операций малой арифметики                               | 6         |
| 1.3 Примеры вычислений                                              | 6         |
| <b>2 Особенности реализации</b>                                     | <b>7</b>  |
| 2.1 Структура проекта                                               | 7         |
| 2.2 Структуры данных                                                | 7         |
| 2.2.1 Класс SmallArithmetic                                         | 7         |
| 2.2.2 Класс BigArithmeticCalc                                       | 8         |
| 2.3 Класс SmallArithmetic                                           | 8         |
| 2.3.1 Назначение и структура данных                                 | 8         |
| 2.3.2 Метод nextSymbol(const string& current)                       | 9         |
| 2.3.3 Метод compareSymbols(const string& a, const string& b)        | 10        |
| 2.3.4 Метод addByHasse(const string& a, const string& b)            | 11        |
| 2.3.5 Метод multiplyByHasse(const string& a, const string& b)       | 12        |
| 2.3.6 Метод subtractByHasse(const string& a, const string& b)       | 13        |
| 2.3.7 Метод buildAdditionTableWithCarry()                           | 14        |
| 2.4 Класс BigArithmeticCalc                                         | 15        |
| 2.4.1 Описание и структура данных                                   | 15        |
| 2.4.2 Метод removeSign(const string& num)                           | 16        |
| 2.4.3 Метод addSign(const string& num, bool negative)               | 17        |
| 2.4.4 Метод deleteTrashZeros(const string& num)                     | 17        |
| 2.4.5 Метод compareBigUnsigned(const string& a, const string& b)    | 18        |
| 2.4.6 Метод addBigUnsigned(const string& a, const string& b)        | 19        |
| 2.4.7 Метод subtractBigUnsigned(const string& a, const string& b)   | 20        |
| 2.4.8 Метод multiplyByDigit(const string& num, const string& digit) | 22        |
| 2.4.9 Метод multiplyBigUnsigned(const string& a, const string& b)   | 23        |
| 2.4.10 Метод divideBigUnsigned(const string& a, const string& b)    | 24        |
| 2.4.11 Метод add(const string& a, const string& b)                  | 26        |
| 2.4.12 Метод subtract(const string& a, const string& b)             | 28        |
| 2.4.13 Метод multiply(const string& a, const string& b)             | 29        |
| 2.4.14 Метод divide(const string& a, const string& b)               | 29        |
| <b>3 Результаты работы программы</b>                                | <b>31</b> |
| 3.1 Запуск программы                                                | 31        |
| 3.2 Сценарий 1: Просмотр справки                                    | 32        |
| 3.2.1 Команда help                                                  | 32        |
| 3.2.2 Команда info                                                  | 32        |

|       |                                                   |    |
|-------|---------------------------------------------------|----|
| 3.3   | Сценарий 2: Просмотр диаграммы Хассе . . . . .    | 33 |
| 3.3.1 | Команда <code>hasse</code> . . . . .              | 33 |
| 3.4   | Сценарий 3: Действия с большими числами . . . . . | 33 |
| 3.4.1 | Сложение положительных чисел . . . . .            | 33 |
| 3.4.2 | Сложение с переносами . . . . .                   | 33 |
| 3.4.3 | Сложение с отрицательными числами . . . . .       | 34 |
| 3.4.4 | Вычитание положительных чисел . . . . .           | 34 |
| 3.4.5 | Вычитание с заимствованием . . . . .              | 34 |
| 3.4.6 | Умножение положительных чисел . . . . .           | 35 |
| 3.4.7 | Умножение с отрицательными числами . . . . .      | 35 |
| 3.4.8 | Деление положительных чисел . . . . .             | 36 |
| 3.4.9 | Деление с отрицательными числами . . . . .        | 36 |
| 3.5   | Сценарий 4: Обработка ошибок . . . . .            | 36 |
| 3.5.1 | Некорректный формат числа . . . . .               | 36 |
| 3.5.2 | Деление на ноль . . . . .                         | 37 |
| 3.5.3 | Переполнение разрядов . . . . .                   | 37 |
| 3.5.4 | Неизвестная команда . . . . .                     | 37 |
| 3.6   | Выход из программы . . . . .                      | 38 |

|                   |           |
|-------------------|-----------|
| <b>Заключение</b> | <b>39</b> |
|-------------------|-----------|

|                          |           |
|--------------------------|-----------|
| <b>Список литературы</b> | <b>41</b> |
|--------------------------|-----------|

# Введение

В ходе курсовой работы требуется сформировать калькулятор «большой» конечной арифметики  $\langle Z_8; +, * \rangle$  для четырех действий  $(+, -, *, \div)$  на основе «малой» конечной арифметики, где задано правило «+1» и выполняются свойства коммутативности  $(+, *)$ , ассоциативности  $(+, *)$ , дистрибутивности  $*$  относительно  $+$ , заданы аддитивная единица «a» и мультипликативная единица «b», а также выполняется свойство: для  $(\forall x) x * a = a$ .

Правило «+1» определяет переход от текущего символа к следующему и задано в соответствии с таблицей 1.

Таблица 1: Правило «+1»

| $x$     | a | b | c | d | e | f | g | h |
|---------|---|---|---|---|---|---|---|---|
| $x + 1$ | b | c | e | g | d | h | f | a |

# 1. Математическое описание

## 1.1. Алгебраические структуры

Множество  $Z$  вместе с набором операций  $\Sigma = \{\varphi_1, \dots, \varphi_m\}$ ,  $\varphi_i : Z^{n_i} \rightarrow Z$ , где  $n_i$  - аргументность операции  $\varphi_i$ , называется алгебраической структурой, универсальной алгеброй или просто алгеброй.

Коммутативное кольцо с единицей - алгебраическая структура  $\langle Z; +, * \rangle$ , которая обладает следующими свойствами:

1.  $(a + b) + c = a + (b + c)$  (сложение ассоциативно)
2.  $\exists 0 \in Z (\forall a \in Z : a + 0 = 0 + a = a)$  (существует нуль)
3.  $\forall a \in Z \exists (-a) \in Z : a + (-a) = 0$  (существует обратный элемент)
4.  $a + b = b + a$  (сложение коммутативно)
5.  $(a * b) * c = a * (b * c)$  (умножение ассоциативно)
6.  $a * (b + c) = a * b + a * c$  (умножение дистрибутивно относительно сложения)
7.  $a * b = b * a$  (умножение коммутативно)
8.  $\exists 1 \in Z (\forall a \in Z : a * 1 = 1 * a = a)$  (существует единица)

«Малая» конечная арифметика - это коммутативное кольцо с единицей  $\langle Z_8; +, * \rangle$ , элементы которого являются одноразрядными символами. В данной работе используется множество  $Z_8 = \{a, b, c, d, e, f, g, h\}$ .

«Большая» конечная арифметика - это коммутативное кольцо с единицей  $\langle Z_8^n; +, * \rangle$ , элементами которого являются слова длины  $n$ , составленные из символов множества  $Z_8$ , в котором операции сложения и умножения выполняются позиционно с учётом переносов. В данной работе  $n = 8$ , поэтому итоговая структура имеет вид  $\langle Z_8^8; +, * \rangle$ .

## 1.2. Таблицы операций малой арифметики

Введем отношение порядка в «малой» конечной арифметике:

$$a \prec b \prec c \prec e \prec d \prec g \prec f \prec h$$

На рисунках 1 и 2 представлена таблица переносов для сложения и умножения соответственно

|   | a | b | c | d | e | f | g | h |
|---|---|---|---|---|---|---|---|---|
| a | a | b | c | d | e | f | g | h |
| b | b | c | e | g | d | h | f | a |
| c | c | e | d | f | g | a | h | b |
| d | d | g | f | a | h | c | b | e |
| e | e | d | g | h | f | b | a | c |
| f | f | h | a | c | b | d | e | g |
| g | g | f | h | b | a | e | c | d |
| h | h | a | b | e | c | g | d | f |

|   | a | b | c | d | e | f | g | h |
|---|---|---|---|---|---|---|---|---|
| a | a | a | a | a | a | a | a | a |
| b | a | a | a | a | a | a | a | b |
| c | a | a | a | a | a | b | a | b |
| d | a | a | a | b | a | b | b | b |
| e | a | a | a | a | a | b | b | b |
| f | a | a | b | b | b | b | b | b |
| g | a | a | a | b | b | b | b | b |
| h | a | b | b | b | b | b | b | b |

Рис. 1: Таблица сложения и таблица переносов для малой арифметики

|   | a | b | c | d | e | f | g | h |
|---|---|---|---|---|---|---|---|---|
| a | a | a | a | a | a | a | a | a |
| b | a | b | c | d | e | f | g | h |
| c | a | c | d | a | f | d | c | f |
| d | a | d | a | a | d | a | d | d |
| e | a | e | f | d | b | c | h | g |
| f | a | f | d | a | c | d | f | c |
| g | a | g | c | d | h | f | b | e |
| h | a | h | f | d | g | c | e | b |

|   | a | b | c | d | e | f | g | h |
|---|---|---|---|---|---|---|---|---|
| a | a | a | a | a | a | a | a | a |
| b | a | a | a | a | a | a | a | a |
| c | a | a | a | b | a | b | b | b |
| d | a | a | b | c | b | e | c | e |
| e | a | a | a | b | b | c | b | c |
| f | a | a | b | e | c | d | e | g |
| g | a | a | b | c | b | e | e | d |
| h | a | a | b | e | c | g | d | f |

Рис. 2: Таблица умножения и таблица переносов для малой арифметики

## 1.3. Примеры вычислений

- $e + d = e + c + c = e + b + b + b + b = h$
- $g + c = g + b + b = f + b = h$
- $h + f = h + c + d = h + b + b + c + c = h + b + b + b + b + b + b = g$
- $d * c = d * (b + b) = d + d = a$
- $g * f = g * (g + b) = g * g + g = g * (b + d) = g + g + g * d = g + g + g * (b + e) = g + g + g + g * e = g + g + g + g * (c + b) = g + g + g + g + g * c = g + g + g + g + g * (b + b) = g + g + g + g + g + g = c + c + c = d + c = f$

## 2. Особенности реализации

### 2.1. Структура проекта

Проект имеет следующую структуру:

- `SmallArithmetic.hpp` и `SmallArithmetic.cpp`: содержат реализацию малой конечной арифметики на основе диаграммы Хассе. Класс выполняет операции над однозначными элементами алфавита.
- `BigArithmeticCalc.hpp` и `BigArithmeticCalc.cpp`: содержат реализацию большой конечной арифметики. Класс выполняет действия столбиком над многозначными числами.
- `UI.hpp` и `UI.cpp`: реализуют пользовательский интерфейс, обработку команд и арифметических выражений.
- `config.hpp`: содержит конфигурацию системы - мощность алфавита, правило "+1" и сам алфавит.
- `main.cpp`: точка входа в программу.
- `CMakeLists.txt`: автоматизация сборки проекта.

### 2.2. Структуры данных

Для реализации конечной арифметики используются два класса: `SmallArithmetic` и `BigArithmeticCalc`.

#### 2.2.1. Класс `SmallArithmetic`

Класс хранит следующие основные структуры:

- `map<string, string> plusOneRule`: правило перехода к следующему элементу алфавита.
- `map<string, string> inverseMap`: таблица обратных элементов для деления.
- `map<tuple<string, string, string>, pair<string, string>> additionTableWithCarry`: таблица сложения с переносом.
- `vector<vector<string>> addTable, mulTable, subTable, divTable`: таблицы операций малой арифметики.
- `int N`: мощность алфавита.
- `static const int MAX_DIGITS = 8`: максимальное количество разрядов в числах.

### 2.2.2. Класс BigArithmeticCalc

Класс содержит:

- `SmallArithmetic small`: объект малой арифметики для выполнения посимвольных операций.

## 2.3. Класс SmallArithmetic

### 2.3.1. Назначение и структура данных

Класс `SmallArithmetic` реализует малую конечную арифметику на заданном алфавите с правилом "+1". Все действия выполняются через построение диаграммы Хассе.

```
1 class SmallArithmetic {
2 private:
3     int N;
4     map<string, string> plusOneRule;
5     vector<string> alphabet;
6
7     map<string, string> inverseMap;
8
9     map<tuple<string, string, string>, pair<string, string>>
10    additionTableWithCarry;
11    map<string, string> negationMap;
12
13    vector<vector<string>> addTable;
14    vector<vector<string>> mulTable;
15    vector<vector<string>> subTable;
16
17    string additiveIdentity;
18    string multiplicativeIdentity;
19    string universum;
20    string emptySet;
21
22    static const int MAX_DIGITS = 8;
23
24    string addByHasse(const string& a, const string& b) const;
25    string multiplyByHasse(const string& a, const string& b) const;
26    string subtractByHasse(const string& a, const string& b) const;
27
28    optional<string> findMultiplicativeInverse(const string& x);
29    void buildInverseMap();
30
31    void buildAddTable();
32    void buildMulTable();
33    void buildSubTable();
34
35    void buildAdditionTableWithCarry();
```

```

36     bool checkCarry(const string& start, const string& steps)
const;
37     void buildNegationMap();
38
39     string nextSymbol(const string& current) const;
40     int compareSymbols(const string& a, const string& b) const;
41
42     void printTable(const vector<vector<string>>& table) const;
43
44 public:
45     SmallArithmetic(int n,
46                     const map<string, string>& rule,
47                     const vector<string>& alph,
48                     const string& addId = "a",
49                     const string& mulId = "b");
50
51     const vector<string>& getAlphabet() const;
52     int getN() const;
53
54     const string& getAdditiveIdentity() const;
55     const string& getMultiplicativeIdentity() const;
56     const string& getUniversum() const;
57     const string& getEmptySet() const;
58     int getMaxDigits() const;
59     string getMinNumber() const;
60     string getMaxNumber() const;
61
62     bool isValidElement(const string& elem) const;
63     int compareElems(const string& a, const string& b) const;
64     string nextElem(const string& current) const;
65     pair<string, string> addWithCarry(const string& c1,
66                                     const string& c2,
67                                     const string& carry_in)
const;
68     string smallSubtract(const string& a, const string& b) const;
69
70     void printAddTable() const;
71     void printMulTable() const;
72     void printSubTable() const;
73     void printAllTables() const;
74     void printHasseDiagram() const;
75 };

```

Listing 1: Интерфейс класса SmallArithmetic

## Ключевые методы

### 2.3.2. Метод nextSymbol(const string& current)

**Описание:** Возвращает следующий элемент алфавита согласно правилу "+1".  
Используется для обхода диаграммы Хассе при выполнении операций сложения

ния и умножения.

**Вход:**

- `const string& current`: Текущий символ алфавита.

**Выход:**

- `string`: Следующий символ по правилу "+1".

```
1 string SmallArithmetic::nextSymbol(const string& current) const {
2     auto it = plusOneRule.find(current);
3     if (it == plusOneRule.end()) {
4         throw runtime_error("invalid symbol: " + current);
5     }
6     return it->second;
7 }
```

Listing 2: Реализация `nextSymbol()`

### 2.3.3. Метод `compareSymbols(const string& a, const string& b)`

**Описание:** Сравнивает два элемента алфавита, определяя их относительный порядок в диаграмме Хассе.

**Вход:**

- `const string& a`: Первый элемент для сравнения.
- `const string& b`: Второй элемент для сравнения.

**Выход:**

- `int`: 0 (если  $a = b$ ), -1 (если  $a < b$ ), 1 (если  $a > b$ ) флаг сравнения.

```
1 int SmallArithmetic::compareSymbols(const string& a, const string&
2     b) const {
3     if (a == b) return 0;
4     string current = additiveIdentity;
5     while (true) {
6         if (current == a) return -1;
7         if (current == b) return 1;
8         current = nextSymbol(current);
9         if (current == additiveIdentity) break;
10    }
11    return 0;
12 }
```

Listing 3: Реализация `compareSymbols()`

Алгоритм последовательно проходит по циклу правила "+1", начиная с нейтрального элемента, и определяет, какой из элементов встречается раньше.

### 2.3.4. Метод `addByHasse(const string& a, const string& b)`

**Описание:** Выполняет сложение двух элементов алфавита путём последовательного применения правила "+1".

Результат вычисляется как  $a + b = a + \underbrace{1 + 1 + \dots + 1}_b$ .

**Вход:**

- `const string& a`: Первое слагаемое.
- `const string& b`: Второе слагаемое.

**Выход:**

- `string`: Результат сложения  $a + b$ .

```
1 string SmallArithmetic::addByHasse(const string& a, const string&
  b) const {
2     if (b == additiveIdentity) return a;
3
4     string counter = additiveIdentity;
5     string result = a;
6
7     while (counter != b) {
8         result = nextSymbol(result);
9         counter = nextSymbol(counter);
10    }
11
12    return result;
13 }
```

Listing 4: Реализация `addByHasse()`

Алгоритм работает следующим образом:

1. Если второе слагаемое является нейтральным элементом, возвращается первое слагаемое;
2. Инициализируется счётчик значением нейтрального элемента;
3. В цикле счётчик увеличивается до значения второго слагаемого;
4. Одновременно результат увеличивается на ту же величину;
5. Возвращается итоговое значение.

### 2.3.5. Метод multiplyByHasse(const string& a, const string& b)

**Описание:** Выполняет умножение двух элементов алфавита через повторное сложение. Результат вычисляется как  $a \times b = \underbrace{a + a + \dots + a}_b$ .

**Вход:**

- const string& a: Множитель.
- const string& b: Множитель.

**Выход:**

- string: Результат умножения  $a \times b$ .

```
1 string SmallArithmetic::multiplyByHasse(const string& a, const
  string& b) const {
2     // специальный случай:  $a * a =$  универсум
3     if (a == additiveIdentity && b == additiveIdentity) {
4         return universum;
5     }
6
7     if (a == additiveIdentity || b == additiveIdentity) {
8         return additiveIdentity;
9     }
10
11     string counter = additiveIdentity;
12     string result = additiveIdentity;
13
14     // буквенный счетчик
15     while (counter != b) {
16         result = addByHasse(result, a);
17         counter = nextSymbol(counter);
18     }
19
20     return result;
21 }
```

Listing 5: Реализация multiplyByHasse()

**Алгоритм:**

1. Если имеем произведение двух нейтральных элементов по сложению получаем универсум;
2. Если хотя бы один множитель - нейтральный элемент, результат - нейтральный элемент;
3. Иначе выполняется повторное сложение множимого с самим собой  $b$  раз.

### 2.3.6. Метод `subtractByHasse(const string& a, const string& b)`

**Описание:** Выполняет вычитание путём поиска такого элемента  $c$ , что  $b + c = a$ .

**Вход:**

- `const string& a`: Уменьшаемое.
- `const string& b`: Вычитаемое.

**Выход:**

- `string`: Результат вычитания  $a - b$  или строка ошибки.

```
1 string SmallArithmetic::subtractByHasse(const string& a, const
  string& b) const {
2     for (const auto& candidate : alphabet) {
3         if (addByHasse(b, candidate) == a) {
4             return candidate;
5         }
6     }
7     return "ERR: with subtract";
8 }
```

Listing 6: Реализация `subtractByHasse()`

Алгоритм перебирает все элементы алфавита и проверяет, какой из них при сложении с вычитаемым даёт уменьшаемое.

### 2.3.7. Метод buildAdditionTableWithCarry()

**Описание:** Строит таблицу для сложения с учётом входящего переноса. Таблица используется в большой арифметике при сложении столбиком. Для каждой тройки  $(a, b, c_{in})$  вычисляется пара  $(sum, c_{out})$ .

**Вход:**

- `vector<string> alphabet`

**Выход:**

- `map<tuple<string, string, string>, pair<string, string>>`  
additionTableWithCarry - таблица для сложения с учётом входящего переноса.

```
1 void SmallArithmetic::buildAdditionTableWithCarry() {
2     for (const auto& c1 : alphabet) {
3         for (const auto& c2 : alphabet) {
4             for (const auto& carry_in : {additiveIdentity,
multiplicativeIdentity}) {
5                 string sum1 = addByHasse(c1, c2); // сумма без
переноса
6                 string final_sum = addByHasse(sum1, carry_in); //
финальная сумма с учетом переноса
7
8                 bool carry1 = checkCarry(c1, c2); // проверяем
перенос c1 + c2
9                 bool carry2 = checkCarry(sum1, carry_in); //
проверяем перенос sum1 + carry_in
10
11                 string carry_out = (carry1 || carry2) ?
multiplicativeIdentity : additiveIdentity; // выставляем перенос
12
13                 additionTableWithCarry[make_tuple(c1, c2,
carry_in)] = make_pair(final_sum, carry_out);
14             }
15         }
16     }
17 }
```

Listing 7: Реализация buildAdditionTableWithCarry()

Таблица содержит тройки  $(a, b, c_{in})$  и соответствующие им пары  $(sum, c_{out})$ , где:

- $a, b$  - слагаемые разрядов;
- $c_{in}$  - входящий перенос из младшего разряда;
- $sum$  - результат сложения в текущем разряде;
- $c_{out}$  - исходящий перенос в старший разряд.

## 2.4. Класс BigArithmeticCalc

### 2.4.1. Описание и структура данных

Класс BigArithmeticCalc реализует большую конечную арифметику - действия над многозначными числами. Внутри использует класс SmallArithmetic для посимвольных операций и действий.

```
1 class BigArithmeticCalc {
2 private:
3     SmallArithmetic small;
4
5     bool isNegative(const string& num) const;
6     string removeSign(const string& num) const;
7     string addSign(const string& num, bool negative) const;
8
9     bool isValidNumber(const string& num) const;
10
11     string deleteTrashZeros(const string& num) const;
12
13     bool isOverflow(const string& num) const;
14
15     string addBigUnsigned(const string& a, const string& b) const;
16     string subtractBigUnsigned(const string& a, const string& b)
17 const;
18     string multiplyBigUnsigned(const string& a, const string& b)
19 const;
20     pair<string, string> divideBigUnsigned(const string& a, const
21 string& b) const;
22
23     string multiplyByDigit(const string& num, const string& digit)
24 const;
25
26     int compareBigUnsigned(const string& a, const string& b) const;
27
28     static const int MAX_DIGITS = 8;
29 public:
30     BigArithmeticCalc(int n,
31                       const map<string, string>& rule,
32                       const vector<string>& alph,
33                       const string& addId = "a",
34                       const string& mulId = "b");
35
36     const vector<string>& getAlphabet() const;
37     int getMaxDigits() const;
38     string getMinNumber() const;
39     string getMaxNumber() const;
40
41     string add(const string& a, const string& b) const;
42     string multiply(const string& a, const string& b) const;
43     string subtract(const string& a, const string& b) const;
44     string divide(const string& a, const string& b) const;
```

```

42 void printAddTable() const;
43 void printMulTable() const;
44 void printSubTable() const;
45 void printAllTables() const;
46 void printInfo() const;
47 void printHasseDiagram() const;
48 void printHelp() const;
49 };

```

Listing 8: Интерфейс класса BigArithmeticCalc

## Вспомогательные методы

### 2.4.2. Метод removeSign(const string& num)

**Описание:** Удаляет знак минус из начала строки, возвращая абсолютное значение числа.

**Вход:**

- const string& num: Число.

**Выход:**

- string: Число без знака.

```

1 string BigArithmeticCalc::removeSign(const string& num) const {
2     if (isNegative(num)) {
3         return num.substr(1);
4     }
5     return num;
6 }

```

Listing 9: Реализация removeSign()

### 2.4.3. Метод addSign(const string& num, bool negative)

**Описание:** Добавляет знак минус к числу, если указан флаг отрицательности. Нейтральный элемент по сложению всегда возвращается без знака.

**Вход:**

- `const string& num`: Число без знака.
- `bool`: Флаг отрицательности.

**Выход:**

- `string`: Число со знаком (если `negative = true`) или без знака.

```
1 string BigArithmeticCalc::addSign(const string& num, bool
   negative) const {
2     if (num == small.getAdditiveIdentity()) return num;
3     return negative ? "-" + num : num;
4 }
```

Listing 10: Реализация addSign()

### 2.4.4. Метод deleteTrashZeros(const string& num)

**Описание:** Удаляет ведущие нули из числа. Если число состоит только из нулей, возвращает нейтральный элемент по сложению.

**Вход:**

- `const string& num`: Число с возможными ведущими нулями.

**Выход:**

- `string`: Число без ведущих нулей.

```
1 string BigArithmeticCalc::deleteTrashZeros(const string& num)
   const {
2     if (num.empty()) return small.getAdditiveIdentity();
3
4     bool negative = isNegative(num);
5     string unsign_num = removeSign(num);
6
7     size_t firstNonZero = 0;
8     while (firstNonZero < unsign_num.length() &&
9            string(1, unsign_num[firstNonZero]) ==
10    small.getAdditiveIdentity()) {
11         firstNonZero++;
12     }
13
14     if (firstNonZero == unsign_num.length()) {
15         return small.getAdditiveIdentity();
16     }
17 }
```

```

16
17     string result = unsign_num.substr(firstNonZero);
18
19     return addSign(result, negative);
20 }

```

Listing 11: Реализация deleteTrashZeros()

Алгоритм:

1. Удаляется знак (если есть);
2. Подсчитывается количество ведущих нулей;
3. Если все символы - нули, возвращается нейтральный элемент;
4. Иначе возвращается подстрока без ведущих нулей со знаком (если необходимо).

#### 2.4.5. Метод compareBigUnsigned(const string& a, const string& b)

**Описание:** Сравнивает два беззнаковых многозначных числа.

**Вход:**

- const string& a: Первое число.
- const string& b: Второе число.

**Выход:**

- int: 0 (если  $a = b$ ), 1 (если  $a > b$ ), -1 (если  $a < b$ ).

```

1 int BigArithmeticCalc::compareBigUnsigned(const string& a, const
  string& b) const {
2     string na = deleteTrashZeros(a);
3     string nb = deleteTrashZeros(b);
4
5     if (na.length() != nb.length()) {
6         return na.length() > nb.length() ? 1 : -1;
7     }
8
9     for (size_t i = 0; i < na.length(); ++i) {
10        int cmp = small.compareElems(string(1, na[i]), string(1,
  nb[i]));
11        if (cmp != 0) return cmp;
12    }
13
14    return 0;
15 }

```

Listing 12: Реализация compareBigUnsigned()

Алгоритм:

1. Нормализация чисел (удаление ведущих нулей);
2. Сравнение по количеству разрядов;
3. При равной длине, посимвольное сравнение слева направо.

## Методы реализации действий большой арифметики

### 2.4.6. Метод `addBigUnsigned(const string& a, const string& b)`

**Описание:** Выполняет сложение двух беззнаковых многозначных чисел столбиком с учётом переноса между разрядами.

**Вход:**

- `const string& a`: Первое слагаемое.
- `const string& b`: Второе слагаемое.

**Выход:**

- `string`: Сумма  $a + b$  без ведущих нулей.

```
1 string BigArithmeticCalc::addBigUnsigned(const string& a, const
  string& b) const {
2     if (!isValidNumber(a) || !isValidNumber(b)) {
3         return "ERR: invalid number";
4     }
5
6     string num1 = a;
7     string num2 = b;
8
9     // добавляем нули
10    size_t max_len = max(num1.length(), num2.length());
11    while (num1.length() < max_len) {
12        num1 = small.getAdditiveIdentity() + num1;
13    }
14    while (num2.length() < max_len) {
15        num2 = small.getAdditiveIdentity() + num2;
16    }
17
18    string result;
19    string carry = small.getAdditiveIdentity();
20
21    for (int i = static_cast<int>(max_len) - 1; i >= 0; --i) {
22        string digit1(1, num1[i]);
23        string digit2(1, num2[i]);
24
25        auto res = small.addWithCarry(digit1, digit2, carry);
26        if (res.first == "ERR") {
```

```

27         return "ERR: invalid addition";
28     }
29
30     string sum = res.first;
31     carry = res.second;
32
33     result = sum + result;
34 }
35
36 if (carry != small.getAdditiveIdentity()) {
37     result = carry + result;
38 }
39
40 if (isOverflow(result)) {
41     return "ERR: overflow";
42 }
43
44 return deleteTrashZeros(result);
45 }

```

Listing 13: Реализация addBigUnsigned()

#### Алгоритм

1. Проверка валидности входных строк;
2. Выравнивание чисел по длине путём добавления ведущих нулей к более короткому;
3. Поразрядное сложение справа налево с учётом переноса;
4. Добавление финального переноса в старший разряд при его наличии;
5. Проверка результата на переполнение;
6. Удаление ведущих нулей из результата.

#### 2.4.7. Метод subtractBigUnsigned(const string& a, const string& b)

**Описание:** Выполняет вычитание двух беззнаковых многозначных чисел столбиком с учётом заимствования из старших разрядов. Требуется, чтобы  $a \geq b$ .

##### Вход:

- const string& a: Уменьшаемое ( $a \geq b$ ).
- const string& b: Вычитаемое.

##### Выход:

- string: Разность  $a - b$  без ведущих нулей.

```

1 string BigArithmeticCalc::subtractBigUnsigned(const string& a,
2 const string& b) const {
3     if (!isValidNumber(a) || !isValidNumber(b)) {
4         return "ERR: invalid number";
5     }
6
7     // сравниваем числа
8     int cmp = compareBigUnsigned(a, b);
9     if (cmp == 0) {
10        return small.getAdditiveIdentity();
11    }
12
13    // определяем большее и меньшее число
14    string larger = (cmp >= 0) ? a : b;
15    string smaller = (cmp >= 0) ? b : a;
16
17    // выравниваем длины
18    size_t max_len = max(larger.length(), smaller.length());
19    while (larger.length() < max_len) {
20        larger = small.getAdditiveIdentity() + larger;
21    }
22    while (smaller.length() < max_len) {
23        smaller = small.getAdditiveIdentity() + smaller;
24    }
25
26    string result;
27    bool borrow = false; // флаг заема
28
29    // вычитаем справа налево
30    for (int i = static_cast<int>(max_len) - 1; i >= 0; --i) {
31        string current(1, larger[i]);
32        string digit2(1, smaller[i]);
33
34        if (borrow) {
35            if (current == small.getAdditiveIdentity()) {
36                current = small.getAlphabet().back();
37            } else {
38                current = small.smallSubtract(current,
39                small.getMultiplicativeIdentity());
40                borrow = false;
41            }
42        }
43
44        int cmp_digits = small.compareElems(current, digit2);
45
46        if (cmp_digits >= 0) {
47            string result_digit = small.smallSubtract(current,
48            digit2);
49            result = result_digit + result;
50        } else {
51            string temp = current; // аналог 10 + a если (нужен
52            заем)

```

```

49         for (int j = 0; j < small.getN(); j++) {
50             temp = small.nextElem(temp);
51         }
52
53         // теперь вычитаем
54         string result_digit = small.smallSubtract(temp,
digit2);
55         result = result_digit + result;
56         borrow = true; // флаг
57     }
58 }
59 if (borrow) {
60     return "ERR: borrow after subtraction";
61 }
62 return deleteTrashZeros(result);
63 }

```

Listing 14: Реализация subtractBigUnsigned()

Алгоритм:

1. Проверка валидности входных строк;
2. Выравнивание чисел по длине путём добавления ведущих нулей к более короткому;
3. Поразрядное вычитание справа налево с учётом заема;
4. Если текущая цифра меньше вычитаемой, берем заем у старшего разряда и корректируем результат.
5. Формируем итоговую строку результата и убираем ведущие нули.

#### 2.4.8. Метод multiplyByDigit(const string& num, const string& digit)

**Описание:** Умножает многозначное число на однозначное число (один элемент алфавита). В процессе работы последовательно формируются неполные произведения путём многократного суммирования исходного числа. Для определения количества операций сложения используется буквенный счётчик, который увеличивается до тех пор, пока не достигнет значения множителя.

**Вход:**

- const string& num: Многозначное число.
- const string& digit: Однозначное число (элемент алфавита).

**Выход:**

- string: Произведение  $num \times digit$ .

```

1 string BigArithmeticCalc::multiplyByDigit(const string& num, const
  string& digit) const {
2     if (digit == small.getAdditiveIdentity()) return
small.getAdditiveIdentity();
3     if (digit == small.getMultiplicativeIdentity()) return num;
4
5     string result = small.getAdditiveIdentity();
6     string counter = small.getAdditiveIdentity();
7
8     // умножаем через многократное сложение с буквенным счетчиком
9     while (counter != digit) {
10         result = addBigUnsigned(result, num);
11         if (result.substr(0, 4) == "ERR:") return result;
12         counter = small.nextElem(counter);
13     }
14
15     return result;
16 }

```

Listing 15: Реализация multiplyByDigit()

#### 2.4.9. Метод multiplyBigUnsigned(const string& a, const string& b)

**Описание:** Выполняет умножение двух беззнаковых многозначных чисел столбиком. Число  $a$  умножается на каждую цифру числа  $b$ , неполные произведения сдвигаются и суммируются.

**Вход:**

- const string& a: Множитель.
- const string& b: Множитель.

**Выход:**

- string: Произведение  $a \times b$  без ведущих нулей.

```

1 string BigArithmeticCalc::multiplyBigUnsigned(const string& a,
  const string& b) const {
2     if (!isValidNumber(a) || !isValidNumber(b)) {
3         return "ERR: invalid number";
4     }
5
6     if (a == small.getAdditiveIdentity() || b ==
small.getAdditiveIdentity()) {
7         return "[" + getMinNumber() + ";" + getMaxNumber() + "];"
8     }
9
10    string result = small.getAdditiveIdentity();
11
12    // умножение столбиком: проходим по каждой цифре множителя
справа налево

```

```

13     for (int i = b.length() - 1; i >= 0; --i) {
14         string multiplier_digit(1, b[i]);
15
16         if (multiplier_digit != small.getAdditiveIdentity()) {
17             string partial_product = multiplyByDigit(a,
18 multiplier_digit);
19             if (partial_product.substr(0, 4) == "ERR:") return
20 partial_product;
21
22             // сдвиг влево добавляем( нули справа)
23             int shift_count = b.length() - 1 - i;
24             for (int j = 0; j < shift_count; j++) {
25                 partial_product += small.getAdditiveIdentity();
26             }
27
28             result = addBigUnsigned(result, partial_product);
29             if (result.substr(0, 4) == "ERR:") return result;
30         }
31     }
32
33     return deleteTrashZeros(result);
34 }

```

Listing 16: Реализация multiplyBigUnsigned()

Алгоритм:

1. Проверка валидности входных строк;
2. Умножение числа  $a$  на каждую цифру числа  $b$ ;
3. Сдвиг неполные произведений влево на соответствующее количество разрядов;
4. Суммирование всех неполные произведения.
5. Удаление ведущих нулей из результата.

#### 2.4.10. Метод divideBigUnsigned(const string& a, const string& b)

**Описание:** Выполняет деление двух беззнаковых многозначных чисел ”уголком”. Возвращает частное и остаток.

**Вход:**

- const string& a: Делимое.
- const string& b: Делитель.

**Выход:**

- pair<string, string>: частное, остаток.

```

1 pair<string, string> BigArithmeticCalc::divideBigUnsigned(const
  string& a, const string& b) const {
2   if (!isValidNumber(a) || !isValidNumber(b)) {
3     return {"ERR", "ERR"};
4   }
5
6   // деление на ноль
7   if (b == small.getAdditiveIdentity()) {
8     // 0/0 = [min;max]
9     if (a == small.getAdditiveIdentity()) {
10      return {small.getUniverse(), small.getUniverse()};
11    }
12    return {small.getEmptySet(), small.getEmptySet()};
13  }
14
15  // если делимое равно нулю
16  if (a == small.getAdditiveIdentity()) {
17    return {small.getAdditiveIdentity(),
18    small.getAdditiveIdentity()};
19  }
20
21  string dividend = a;
22  string divisor = b;
23
24  // если делимое меньше делителя
25  if (compareBigUnsigned(dividend, divisor) < 0) {
26    return {small.getAdditiveIdentity(), dividend};
27  }
28
29  // деление в столбик
30  string q_str;
31  string curr_r = small.getAdditiveIdentity();
32
33  // проходим по каждой цифре делимого слева направо
34  for (size_t i = 0; i < dividend.length(); i++) {
35    // добавляем следующую цифру к остатку
36    curr_r = curr_r + string(1, dividend[i]);
37    curr_r = deleteTrashZeros(curr_r);
38
39    // ищем максимальную цифру частного
40    string q_digit = small.getAdditiveIdentity();
41
42    // пробуем все возможные цифры от 1 до максимальной
43    string test_digit = small.getMultiplicativeIdentity();
44    while (test_digit != small.getAdditiveIdentity()) {
45      string test_product = multiplyByDigit(divisor,
46      test_digit);
47
48      if (compareBigUnsigned(test_product, curr_r) > 0) {
49        break;
50      }
51    }
52  }
53 }

```

```

50         q_digit = test_digit;
51         test_digit = small.nextElem(test_digit);
52     }
53
54     q_str += q_digit;
55
56     // вычитаем произведение из остатка
57     if (q_digit != small.getAdditiveIdentity()) {
58         string product = multiplyByDigit(divisor, q_digit);
59         curr_r = subtractBigUnsigned(curr_r, product);
60     }
61 }
62
63 string q = deleteTrashZeros(q_str);
64 string r = deleteTrashZeros(curr_r);
65
66 return {q, r};
67 }

```

Listing 17: Реализация divideBigUnsigned()

Алгоритм:

1. Проверка корректности входных данных и обработка случаев: деление на ноль, нулевое делимое;
2. Сравнение делимого и делителя для раннего завершения, если делимое меньше делителя;
3. Инициализация частного и текущего остатка;
4. Последовательный просмотр цифр делимого слева направо с накоплением текущего остатка;
5. Для каждого шага подбор максимальной цифры частного, произведение которой на делитель не превышает текущий остаток;
6. Добавление найденной цифры в частное и вычитание соответствующего произведения из остатка;
7. Удаление ведущих нулей из частного и остатка.

## Публичные действия с учётом знаков

### 2.4.11. Метод add(const string& a, const string& b)

**Описание:** Выполняет сложение двух чисел с учётом знаков. Обработывает случаи сложения чисел с одинаковыми и разными знаками.

**Вход:**

- const string& a: Первое слагаемое.

- `const string& b`: Второе слагаемое.

### Выход:

- `string`: Сумма  $a + b$  со знаком.

```
1 string BigArithmeticCalc::add(const string& a, const string& b)
2     const {
3         bool neg_a = isNegative(a);
4         bool neg_b = isNegative(b);
5
6         string unsign_a = removeSign(a);
7         string unsign_b = removeSign(b);
8
9         // a + b оба (положительные)
10        if (!neg_a && !neg_b) {
11            return addBigUnsigned(unsign_a, unsign_b);
12        }
13
14        // -a + (-b) = -(a + b)
15        if (neg_a && neg_b) {
16            string sum = addBigUnsigned(unsign_a, unsign_b);
17            if (sum.substr(0, 4) == "ERR:") return sum;
18            return addSign(sum, true);
19        }
20
21        // a + (-b) = a - b
22        if (!neg_a && neg_b) {
23            int cmp = compareBigUnsigned(unsign_a, unsign_b);
24            if (cmp >= 0) {
25                return subtractBigUnsigned(unsign_a, unsign_b);
26            } else {
27                string diff = subtractBigUnsigned(unsign_b, unsign_a);
28                return addSign(diff, true);
29            }
30        }
31
32        // -a + b = b - a
33        if (neg_a && !neg_b) {
34            int cmp = compareBigUnsigned(unsign_b, unsign_a);
35            if (cmp >= 0) {
36                return subtractBigUnsigned(unsign_b, unsign_a);
37            } else {
38                string diff = subtractBigUnsigned(unsign_a, unsign_b);
39                return addSign(diff, true);
40            }
41        }
42
43        return "ERR: unknown case";
44    }
```

Listing 18: Реализация `add()`

## 2.4.12. Метод `subtract(const string& a, const string& b)`

**Описание:** Выполняет вычитание двух чисел с учётом знаков. Обрабатывает случаи вычитания чисел с одинаковыми и разными знаками.

**Вход:**

- `const string& a`: Уменьшаемое.
- `const string& b`: Вычитаемое.

**Выход:**

- `string`: Разность  $a - b$  со знаком.

```
1 string BigArithmeticCalc::subtract(const string& a, const string&
   b) const {
2     bool neg_a = isNegative(a);
3     bool neg_b = isNegative(b);
4
5     string ua = removeSign(a);
6     string ub = removeSign(b);
7
8     // a - b оба ( >= 0)
9     if (!neg_a && !neg_b) {
10        int cmp = compareBigUnsigned(ua, ub);
11        if (cmp >= 0) return subtractBigUnsigned(ua, ub);
12        string diff = subtractBigUnsigned(ub, ua);
13        return addSign(diff, true); // отрицательный результат
14    }
15
16    // a - (-b) = a + b
17    if (!neg_a && neg_b) {
18        return addBigUnsigned(ua, ub);
19    }
20
21    // -a - b = -(a + b)
22    if (neg_a && !neg_b) {
23        string sum = addBigUnsigned(ua, ub);
24        if (sum.substr(0, 4) == "ERR:") return sum;
25        return addSign(sum, true);
26    }
27
28    // -a - (-b) = b - a
29    if (neg_a && neg_b) {
30        int cmp = compareBigUnsigned(ua, ub);
31        if (cmp > 0) {
32            string diff = subtractBigUnsigned(ua, ub);
33            return addSign(diff, true); // -(a - b)
34        } else if (cmp < 0) {
35            string diff = subtractBigUnsigned(ub, ua);
36            return diff;
37        }
38    }
```

```

38     return small.getAdditiveIdentity();
39 }
40
41 return "ERR: unreachable case subtract()";
42 }

```

Listing 19: Реализация subtract()

### 2.4.13. Метод multiply(const string& a, const string& b)

**Описание:** Выполняет умножение двух чисел с учётом знаков. Знак результата определяется правилом:  $(-)\times(-) = (+)$ ,  $(+)\times(-) = (-)$ ,  $(+)\times(+) = (+)$ .

**Вход:**

- const string& a: Множимое.
- const string& b: Множитель.

**Выход:**

- string: Произведение  $a \times b$  со знаком.

```

1 string BigArithmeticCalc::multiply(const string& a, const string&
   b) const {
2     bool neg_a = isNegative(a);
3     bool neg_b = isNegative(b);
4
5     string unsign_a = removeSign(a);
6     string unsign_b = removeSign(b);
7
8     string product = multiplyBigUnsigned(unsign_a, unsign_b);
9     if (product.substr(0, 4) == "ERR:") return product;
10
11     // результат отрицательный, если знаки разные
12     bool result_negative = (neg_a != neg_b);
13
14     return addSign(product, result_negative);
15 }

```

Listing 20: Реализация multiply()

### 2.4.14. Метод divide(const string& a, const string& b)

**Описание:** Выполняет деление двух чисел с учётом знаков. Возвращает частное и остаток.

**Вход:**

- const string& a: Делимое.
- const string& b: Делитель ( $b \neq 0$ ).

## Выход:

- string: Частное  $a/b$  со знаком и остаток.

```
1 string BigArithmeticCalc::divide(const string& a, const string& b)
2     const {
3         bool neg_a = isNegative(a);
4         bool neg_b = isNegative(b);
5
6         string unsign_a = removeSign(a);
7         string unsign_b = removeSign(b);
8
9         auto [q, r] = divideBigUnsigned(unsign_a, unsign_b);
10
11         // обработка специальных случаев
12         if (q == small.getEmptySet() || q == small.getUniverse()) {
13             return "Q: " + q + " | R: " + r;
14         }
15
16         if (q.substr(0, 3) == "ERR") {
17             return q;
18         }
19
20         // если делимое отрицательное, а делитель положительный
21         // то  $-a / b = -(a/b + 1)$ , остаток =  $b - r$ 
22         if (neg_a && !neg_b && r != small.getAdditiveIdentity()) {
23             // добавляем единицу к частному
24             q = addBigUnsigned(q, small.getMultiplicativeIdentity());
25             // вычисляем новый остаток:  $b - r$ 
26             r = subtractBigUnsigned(unsign_b, r);
27             q = addSign(q, true);
28         }
29
30         // если делимое положительное, а делитель отрицательный
31         // то  $a / (-b) = -(a/b)$ , остаток остается  $r$ 
32         else if (!neg_a && neg_b) {
33             q = addSign(q, true);
34         } else if (neg_a && !neg_b) { //  $-a / b = -(a/b)$  без остатка
35             q = addSign(q, true);
36         }
37
38         // если оба отрицательные:  $-a / (-b) = a/b$ 
39         else if (neg_a && neg_b) {}
40
41         return "Q: " + q + " | R: " + r;
42     }
```

Listing 21: Реализация divide()

## 3. Результаты работы программы

### 3.1. Запуск программы

При запуске программа выводит приветственное сообщение и информацию о командах. Пользователь может вводить команды или арифметические выражения.

```
===== Калькулятор БОЛЬШОЙ конечной арифметики Z8 =====  
  
Введите 'help' для справки по командам  
Введите 'info' для информации о системе  
  
===== ИНФОРМАЦИЯ О КОНЕЧНОЙ АРИФМЕТИКЕ =====  
  
размерность: Z8  
аддитивная единица (0): 'a'  
мультипликативная единица (1): 'b'  
  
максимальное количество разрядов: 8  
диапазон чисел: [-hhhhhhhh; hhhhhhhh]  
  
специальные правила:  
•  $a * a = [-hhhhhhhh; hhhhhhhh]$  (универсум)  
•  $\text{число} / a = \emptyset$  (пустое множество)  
•  $a / a = [-hhhhhhhh; hhhhhhhh]$  (диапазон всех чисел)  
• переполнение ( $> 8$  разрядов) = ERR: overflow  
===== ДИАГРАММА ХАССЕ (правило +1) =====  
  
Отношение порядка элементов Z8:  
  
a => b => c => e => d => g => f => h => a (цикл)  
  
=====
```

Рис. 3: Приветственное сообщение при запуске программы

## 3.2. Сценарий 1: Просмотр справки

### 3.2.1. Команда help

При вводе команды `help` программа выводит список всех доступных команд и их описание.

```
===== ПОМОЩЬ =====
доступные команды:

БОЛЬШАЯ АРИФМЕТИКА (до 8 разрядов, с отрицательными):
<число> + <число>      - сложение больших чисел
<число> - <число>      - вычитание больших чисел
<число> * <число>      - умножение больших чисел
<число> / <число>      - деление больших чисел

примечание: отрицательные числа пишутся с '-' (например: -abc)

ИНФОРМАЦИЯ:
info                  - информация об арифметике
hasse                 - диаграмма хассе
tables                - все таблицы операций
add_table             - таблица сложения
mul_table             - таблица умножения
sub_table             - таблица вычитания
div_table             - таблица деления
help                  - эта справка
exit / quit           - выход

примеры:
abd + ghf             - сложение положительных
-abd + ghf            - сложение отрицательного и положительного
hhhhhhh + b           - переполнение (ERR: overFlow)
-c / d                - деление отрицательного на положительное
a / a                 - результат:[-hhhhhhh; hhhhhhhh]
g / a                 - результат: ∅ (пустое множество)

особенности деления с остатком:
• при делении -a / b: частное увеличивается на 1, остаток = b - r
• остаток может быть многозначным

=====
```

Рис. 4: Результат выполнения команды `help`

### 3.2.2. Команда info

При вводе команды `info` программа выводит информацию о конфигурации системы.

```
===== ИНФОРМАЦИЯ О КОНЕЧНОЙ АРИФМЕТИКЕ =====

размерность: Z8
аддитивная единица (0): 'a'
мультипликативная единица (1): 'b'

максимальное количество разрядов: 8
диапазон чисел: [-hhhhhhh; hhhhhh]

специальные правила:
• a * a = [-hhhhhhh; hhhhhh] (универсум)
• число / a = ∅ (пустое множество)
• a / a = [-hhhhhhh; hhhhhh] (диапазон всех чисел)
• переполнение (> 8 разрядов) = ERR: overFlow
```

Рис. 5: Результат выполнения команды `info`

### 3.3. Сценарий 2: Просмотр диаграммы Хассе

#### 3.3.1. Команда `hasse`

При вводе команды `hasse` программа выводит диаграмму Хассе - визуальное представление отношения порядка на алфавите.

```
===== ДИАГРАММА ХАССЕ (правило +1) =====  
  
Отношение порядка элементов Z8:  
  
a => b => c => e => d => g => f => h => a (цикл)  
  
=====
```

Рис. 6: Диаграмма Хассе

Диаграмма показывает циклический порядок элементов алфавита. Каждый элемент связан со следующим по правилу "+1".

### 3.4. Сценарий 3: Действия с большими числами

#### 3.4.1. Сложение положительных чисел

Пользователь вводит выражение для сложения двух положительных чисел.

```
calc> abc + def  
  
===== Результат большой арифметики =====  
abc + def = dga  
=====
```

Рис. 7: Сложение положительных чисел

Результат вычисляется столбиком с учётом переносов между разрядами.

#### 3.4.2. Сложение с переносами

Пример, демонстрирующий механизм переноса в старшие разряды.

```
calc> hhhh + b  
  
===== Результат большой арифметики =====  
hhhh + b = baaaa  
=====
```

Рис. 8: Сложение с переносом

При сложении `hhhh + b` происходит переход через границу алфавита, что вызывает перенос в следующий разряд.

### 3.4.3. Сложение с отрицательными числами

Пример сложения чисел с разными знаками.

```
calc> abc + -abc

===== Результат большой арифметики =====
abc + -abc = a
=====

calc> -def + bcd

===== Результат большой арифметики =====
-def + bcd = -ebc
=====

calc> -bcd + def

===== Результат большой арифметики =====
-bcd + def = ebc
=====
```

Рис. 9: Сложение с отрицательными числами

Программа правильно обрабатывает случаи:

- Сложение противоположных чисел (результат - нейтральный элемент)
- Сложение чисел с разными знаками (вычитание модулей)

### 3.4.4. Вычитание положительных чисел

```
calc> def - abc

===== Результат большой арифметики =====
def - abc = dcd
=====

calc> abc - def

===== Результат большой арифметики =====
abc - def = -dcd
=====
```

Рис. 10: Вычитание положительных чисел

При вычитании меньшего из большего результат положительный, при вычитании большего из меньшего - отрицательный.

### 3.4.5. Вычитание с заимствованием

Пример, демонстрирующий механизм заимствования из старших разрядов.

```

calc> baaa - hhh

===== Результат большой арифметики =====
      baaa - hhh = b
=====

```

Рис. 11: Вычитание с заимствованием

При вычитании  $baaa - hhh$  происходит заимствование из старшего разряда.

#### 3.4.6. Умножение положительных чисел

```

calc> abc * h

===== Результат большой арифметики =====
      abc * h = baf
=====

calc> bc * de

===== Результат большой арифметики =====
      bc * de = gef
=====

```

Рис. 12: Умножение положительных чисел

Умножение выполняется столбиком с промежуточным суммированием.

#### 3.4.7. Умножение с отрицательными числами

```

calc> -bc * de

===== Результат большой арифметики =====
      -bc * de = -gef
=====

calc> -abc * -h

===== Результат большой арифметики =====
      -abc * -h = baf
=====

calc> abc * -h

===== Результат большой арифметики =====
      abc * -h = -baf
=====

```

Рис. 13: Умножение с отрицательными числами

Знак результата определяется правилом:

- $+$   $\times$   $+$   $=$   $+$
- $-$   $\times$   $-$   $=$   $+$
- $+$   $\times$   $-$   $=$   $-$
- $-$   $\times$   $+$   $=$   $-$

### 3.4.8. Деление положительных чисел

```
calc> gef / bc

===== Результат большой арифметики =====
gef / bc = Q: de | R: a
=====

calc> abcd / ef

===== Результат большой арифметики =====
abcd / ef = Q: c | R: ea
=====
```

Рис. 14: Деление положительных чисел

### 3.4.9. Деление с отрицательными числами

```
calc> -gef / bc

===== Результат большой арифметики =====
-gef / bc = Q: -de | R: a
=====

calc> gef / -bc

===== Результат большой арифметики =====
gef / -bc = Q: -de | R: a
=====

calc> -gef / -bc

===== Результат большой арифметики =====
-gef / -bc = Q: de | R: a
=====
```

Рис. 15: Деление с отрицательными числами

## 3.5. Сценарий 4: Обработка ошибок

### 3.5.1. Некорректный формат числа

При вводе числа, содержащего символы вне алфавита, программа выводит сообщение об ошибке.

```

calc> abx / f

===== Результат большой арифметики =====
abx / f = ERR
=====

```

Рис. 16: Ошибка: некорректный формат числа

Символ x не принадлежит алфавиту системы.

### 3.5.2. Деление на ноль

При попытке деления на нейтральный элемент по сложению программа выводит ошибку.

```

calc> abc / a

===== Результат большой арифметики =====
abc / a = Q: ∅ | R: ∅
=====

```

Рис. 17: Ошибка: деление на ноль

### 3.5.3. Переполнение разрядов

При превышении максимального количества разрядов ( $\text{MAX\_DIGITS} = 8$ ) программа выводит ошибку переполнения.

```

calc> hhhhhhhhhhhh * hh

===== Результат большой арифметики =====
hhhhhhhhhhhhh * hh = ERR: overflow
=====

```

Рис. 18: Ошибка: переполнение

Результат действия превышает 8 разрядов, что не допускается системой.

### 3.5.4. Неизвестная команда

При вводе неизвестной команды программа выводит сообщение об ошибке.

```

calc> home
Ошибка: неверный формат команды. Введите 'help' для справки

```

Рис. 19: Ошибка: неизвестная команда

### 3.6. Выход из программы

При вводе команды `exit` или `quit` программа завершает работу с прощальным сообщением.

```
calc> exit  
===== Спасибо за использование калькулятора! =====
```

Рис. 20: Завершение работы программы

## Заключение

В ходе курсовой работы был сформирован калькулятор «большой» конечной арифметики  $\langle Z_8; +, * \rangle$  для четырех действий  $(+, -, *, \div)$  на основе «малой» конечной арифметики, где задано правило «+1» и выполняются свойства коммутативности  $(+, *)$ , ассоциативности  $(+, *)$ , дистрибутивности  $*$  относительно  $+$ , заданы аддитивная единица «а» и мультипликативная единица «b», а также выполняется свойство: для  $(\forall x) x * a = a$ .

## Реализованный функционал

В программной части была реализована малая конечная арифметика с действиями над однозначными элементами через диаграмму Хассе. На основе малой арифметики реализована большая арифметика с действиями над многозначными числами до восьми разрядов. Добавлена поддержка отрицательных чисел с корректной обработкой знаков при выполнении всех действий.

Разработана система команд для просмотра информации о системе, диаграммы Хассе. Реализована обработка ошибок, включая деление на ноль, переполнение разрядов и некорректный формат чисел.

## Достоинства реализации

Использование предвычисленных таблиц действий делает вычисления эффективными: все действия малой арифметики вычисляются один раз при инициализации, а затем используются для быстрого доступа к значениям. Это позволяет избежать повторных вычислений при работе с многозначными числами.

## Недостатки реализации

Конфигурация системы задаётся в файле `config.hpp` и требует перекомпиляции при изменении. Пользователь не может менять параметры во время работы программы.

Также программа ограничена восьмью разрядами, что не позволяет работать с очень большими числами. При превышении лимита возникает ошибка переполнения.

## Масштабирование

Все действий выполняются через консольный интерфейс. Визуализация интерфейса, таблиц и диаграммы Хассе была бы удобнее через GUI при помощи фреймворка Qt.

Можно добавить возможность загружать конфигурацию из файла, чтобы пользователь мог менять алфавит и правило "+1" без повторной компиляции программы.

## Список литературы

- [1] Сайт кафедры с учебными материалами по курсу «Дискретная математика». Ссылка: <https://tema.spbstu.ru/dismath/> (Дата обращения: 21.10.2025).
- [2] Новиков Ф.А. *Дискретная математика*. Учебник. Ссылка на PDF: <https://stugum.wordpress.com/wp-content/uploads/2014/03/novikov.pdf> (Дата обращения: 21.10.2025).
- [3] Кук Д., Бейз Г. Компьютерная математика. - М.,: Наука, 1990. - 384 с. (Дата обращения: 10.12.2025)